



Bilkent University

CS 319

Deliverable-5

S1-T2-Databaseable

Hüsnü Arkca

Yunus Emre Aslan

Mehmet Dizdar

Aral Müftüoğlu

Onur Ege Yakar

Selected Design Pattern 1: Factory Method Pattern

Context: Default Evaluation Criteria Generation for Each Study Type

1. Description

The **Factory Method Pattern** encapsulates object creation in a dedicated “factory” component, instead of letting client code instantiate concrete classes directly. It allows the system to decide at runtime which specific product objects to create, based on some input (e.g., type, configuration), while keeping the caller independent of the concrete classes.

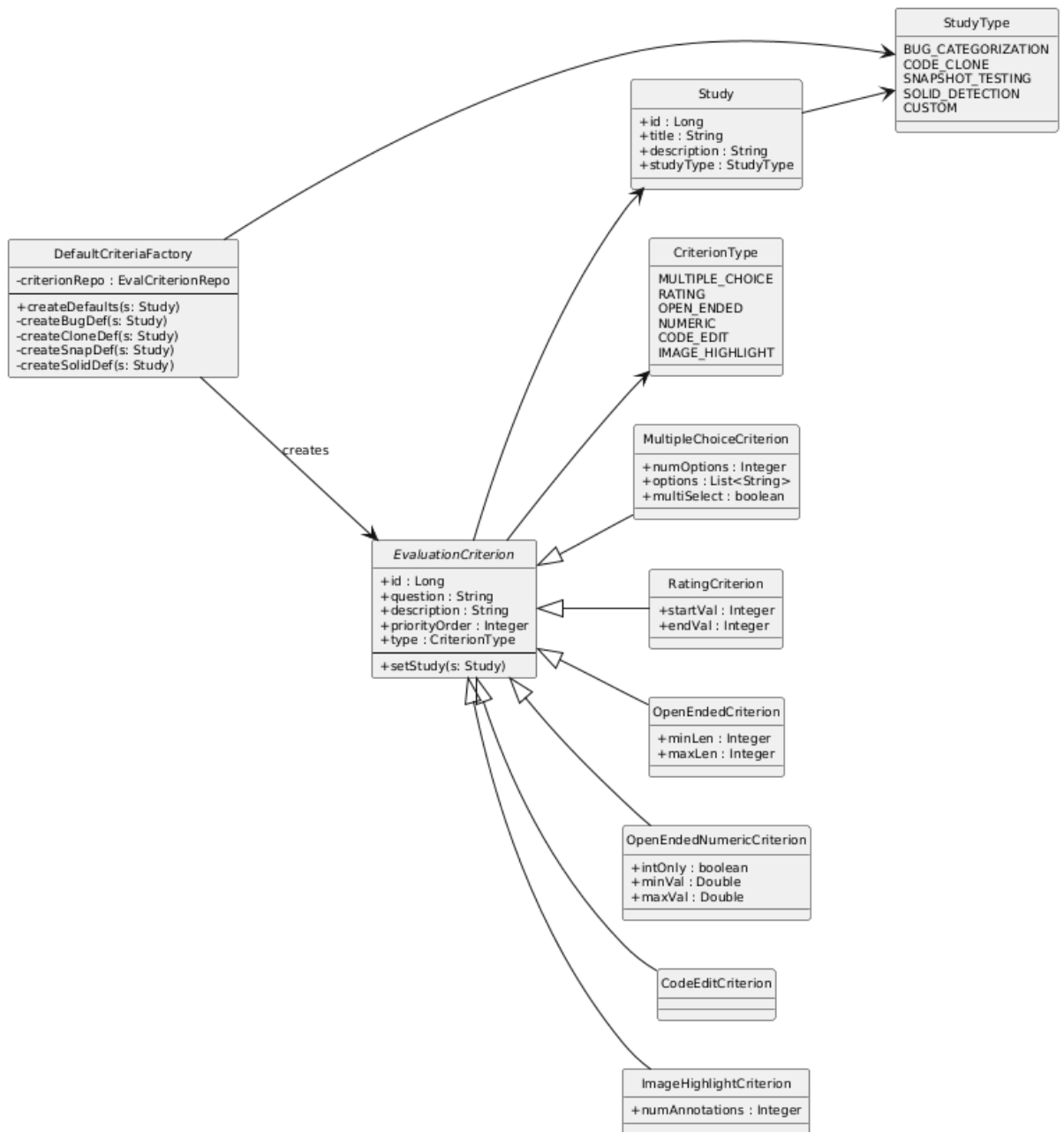
In the Artifact Comparator project, each StudyType requires a different set of EvaluationCriterion objects.

For example:

- BUG_CATEGORIZATION → one multi-select “Bug Category” criterion
- CODE_CLONE → one single-select “Clone Type” criterion
- SNAPSHOT_TESTING → one multi-select “UI Changes” criterion
- SOLID_DETECTION → two criteria (“Violated Principle” and “Difficulty”)
- CUSTOM → no defaults; criteria are defined manually by the researcher on the frontend

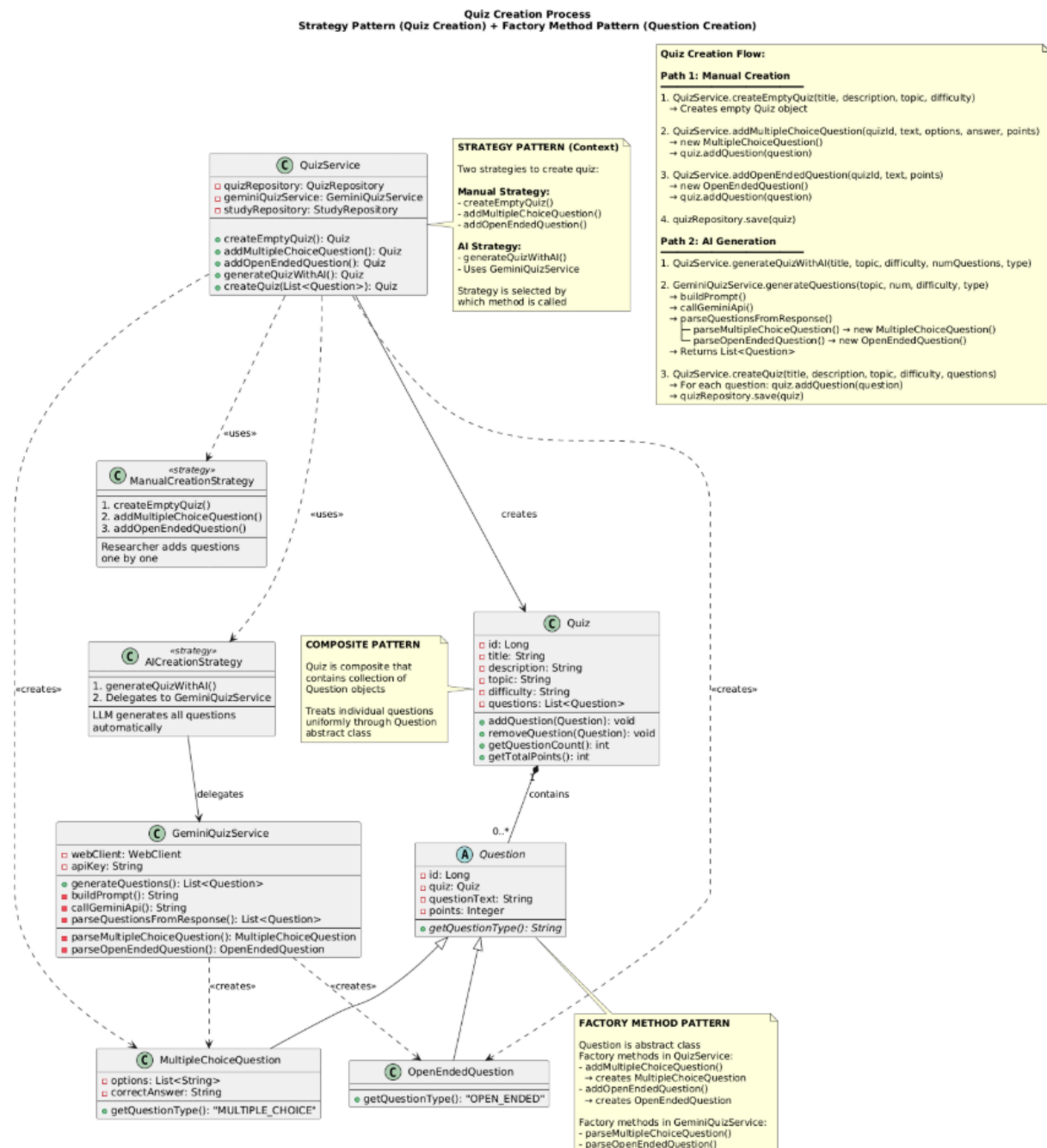
To avoid scattering if/else logic across the codebase, we use a dedicated service class, DefaultCriteriaFactory, which decides **which criterion objects to create** based on the StudyType. Each study type is handled inside the factory’s single creation method (createDefaultsForStudy), where the factory decides which concrete EvaluationCriterion subclasses should be instantiated. The factory centralizes all object-creation logic and prevents this switch-case from being duplicated in controllers or services. This is a concrete use of the **Factory Method Pattern**: the factory encapsulates object creation and hides concrete criterion construction details from the rest of the system.

DefaultCriteriaFactory - Factory Method Pattern with 6 Criterion Types



The Factory Method Pattern is appropriate because the system must create different criterion configurations depending on the selected StudyType, and each configuration involves different EvaluationCriterion subclasses. Without a factory, this logic would be duplicated across controllers or services, violating the Single Responsibility and Open/Closed principles. By centralizing object creation inside DefaultCriteriaFactory, the system becomes easier to maintain and extend. Adding a new study type or criterion type requires only adding a new factory method, without modifying existing client code.

Selected Design Pattern 2: Strategy Pattern



Strategy pattern is used in the main logic of the quiz section of the project. The quiz creation method will be different according to the researcher's choices. QuizService may call GeminiQuizService to use LLM to create the quiz or directly create Question objects with user's inputs.

Also factory method pattern and composite pattern is used. Quiz object contains a list of Question objects but Question is an abstract class. With that implementation, it

is simple to add new question types because no further changes are needed at Quiz class.